SPL Compiler: User Manual (Part 2)

COS341 Project Part 2 · Group 21 · 1st Class submission

1. Group members

Full name	Student number
Antony Van Straten	24590739
Keegan Lewis	24594522
Aidan Dawson	24593542
Sofia Finlayson	24593240

2. Submission overview

This project is submitted as a **1st Class** Part 2 implementation (see Declaration 1). The compiler reads one SPL program from a plain text file and translates it into an executable C++ program. It implements:

- Front-end: lexical analysis, syntax analysis and syntax-tree generation (`tree.xml`)
- Phase 2a: semantic analysis (scopes and name binding)
- Phase 2b: type checking
- Phase 3a: target code generation (`target.cpp`)
- Phase 3b: function and procedure calls, translated by inlining

The compiler is supplied as a runnable Java archive, `group-21.jar`. It has no graphical interface and is run from a terminal. The generated `target.cpp` is then compiled with g++ and run like any other C++ program.

3. Requirements

Software	Needed for
Java 21 or newer	Running the compiler (`group-21.jar`). All libraries it uses are packaged inside the jar.
g++ with C++17 support	Compiling the generated `target.cpp`. Any g++ from version 8 onward supports C++17.

Maven and the project source code are **not** needed to assess the submission. To check both tools, open a terminal (see section 5) and run:

```
java -version
g++ --version
```

The first line of `java -version` must show version 21 or higher, for example `openjdk version "21.0.4"`. If either tool is missing:

- **Java:** install Java 21 (for example Eclipse Temurin 21 from adoptium.net), then open a new terminal.
- **g++ on Windows:** install MinGW-w64 (for example through MSYS2) and add its `bin` folder to the PATH.
- **g++ on macOS:** run `xcode-select --install`. This installs the Command Line Tools, which provide a g++ command.
- **g++ on Linux:** install the `build-essential` package (Debian/Ubuntu) or `gcc-c++` (Fedora).

4. Contents of the ZIP file

File	Description
`group-21.jar`	The executable SPL compiler (runnable Java archive)
`group-21.pdf`	This user manual, including the two signed declarations

5. Running the compiler

1. Extract `group-21.zip` into a folder.
2. Open a terminal in that folder. **Windows:** open the folder in File Explorer, click the address bar, type `cmd` and press Enter. **macOS:** open Terminal (Applications → Utilities → Terminal), type `cd` followed by a space, drag the folder into the Terminal window and press Enter.
3. Run the compiler, giving the SPL test file as the only argument:

```
java -jar group-21.jar test1.txt
```

4. If the program is accepted, compile and run the generated `target.cpp` as described in section 6.
5. Repeat from step 3 for the next test file. Each run first removes the output files of the previous run (see section 7).

Notes on the input file

- The file path can be relative to the folder the terminal is in, or a full path. If the path contains spaces, put it in double quotes, for example `java -jar group-21.jar "C:\SPL Tests\test1.txt"` on Windows or `java -jar group-21.jar "/Users/tutor/SPL Tests/test1.txt"` on macOS.
- The file must be plain text containing one SPL program. Any file name or extension works.
- The end of the file marks the end of the program. The pseudo-symbol `$` is not written in the file.
- Do not double-click `group-21.jar`. It is a console program, so double-clicking opens no window and it will appear that nothing happened.

6. Compiling and running the generated code

Run these commands in the same terminal and folder as the compiler, because `target.cpp` is written there.

macOS and Linux

```
g++ -std=c++17 -o program target.cpp
./program
```

Windows

```
g++ -std=c++17 -o program.exe target.cpp
program.exe
```

The first command compiles `target.cpp` into an executable named `program` (`program.exe` on Windows); the second runs it. What it prints depends entirely on the SPL program that was compiled: every `print` instruction in the SPL source becomes one output statement in `target.cpp`.

Printed values are written one after another, with no spaces or line breaks between them. For example, a program that prints `"zero"`, then `1`, then `2` produces `zero12`.

Compile without `-Wall`. With warnings enabled, g++ reports some labels in `target.cpp` as unused. These labels are emitted on purpose (see section 10) and the warnings are harmless, but they can look like an error.

7. Output files

Both output files are written to the folder the terminal is in when the command is run, which is not necessarily the folder containing the jar. At the start of every run, the compiler deletes any `tree.xml` and `target.cpp` already in that folder, so after a run only the files produced by that run are present. (The one exception is when the input file is not found: the compiler stops before deleting anything.)

File	When it is written
`tree.xml`	As soon as lexical and syntax analysis succeed, before semantic analysis and type checking, so it can exist even when a later stage rejects the program. It is written again once type checking succeeds.
`target.cpp`	Only when every stage succeeds: parsing, semantic analysis, type checking and code generation. After a rejected program, there is no `target.cpp`.

tree.xml

The file has one `root` node (`spl_prog`), `inner` nodes for non-terminals and `leaf` nodes for terminals (the tokens of the program). Every node has a unique `id`; every node except the root has a `parent`; inner nodes list their children by id. A non-terminal that derives the empty string appears as an inner node with an empty `<children/>` element. An excerpt:

```
<syntaxTree>
  <root>
    <id>0</id>
    <contents>spl_prog</contents>
    <children>1</children>
  </root>
  <inner>
    <id>1</id>
    <contents>p</contents>
    <children>2 21 22 165 166</children>
    <parent>0</parent>
  </inner>
  <leaf>
    <id>3</id>
    <contents>#x</contents>
    <parent>2</parent>
  </leaf>
  ...
</syntaxTree>
```

target.cpp

The generated file has two parts:

- **Code-mine:** a single block comment at the top, holding a description of every function and procedure in the SPL program. It is documentation only and is never executed. Any `/*` or `*/` inside it is rewritten as `(*` and `*)`, so that the whole code-mine stays one valid comment.
- **Executable part:** a complete C++ `main` function containing the translated program, with every function call inlined at its call site.

8. Reading the output

The compiler runs its stages in order and stops at the first one that finds an error. A rejected program is a correct result whenever the SPL program really is invalid. Every stage that passed prints a line, so the output shows how far the program got:

- `Parsed OK`, the parse tree and `Check …/tree.xml`: lexical and syntax analysis passed, and `tree.xml` was written.
- `Scope analysis OK`: semantic analysis (scopes and name binding) passed as well.
- `Type analysis OK`: type checking passed, and `tree.xml` was written again.
- `Code generation OK`: `target.cpp` was written. This line only appears when every stage passed.
- `Code generation error`: the program passed every check but could not be translated; no `target.cpp` is written.

Errors from both semantic analysis and type checking start with `Semantic error:`. A type error can be recognised because `Scope analysis OK` is printed directly above it.

Stage	Rejects, for example
Lexical analysis	An illegal symbol, a misspelt keyword, a variable name without `#`, an unterminated string

Stage	Rejects, for example
Syntax analysis	Tokens in an order the SPL grammar does not allow, such as missing braces, a missing `return`, or a loop without `while`/`until`
Semantic analysis	An undeclared variable or function, a duplicate declaration, a recursive call or a call to an outer function
Type checking	A `num` function used as an instruction, a `void` procedure used inside a term, a decimal number in a program that uses `mod` (with `mod`, every number must be an integer), or `mod` and `div` used in the same program
Code generation	A call with the wrong number of arguments

Accepted program

```
Parsed OK: valid4_loop_branch.txt
Parse tree: (spl_prog (p (v_decl #i v_decl) : f_decl : (algo (instr #i (instrTail = (term
0))) ; (algo (instr (loop (cond while) (bool lesser ( (term #i termTail) (term 3) )) do {
(algo (instr (branch if (bool eq ( (term #i termTail) (term 0) )) then { (algo (instr print
(outp "zero")) ; algo) } else { (algo (instr print (outp ( (term #i termTail) ))) ; algo)
})) ; (algo (instr #i (instrTail = (term add ( (term #i termTail) (term 1) )))) ; algo))
})) ; algo))) <EOF>)
Outputting the XML now
Check /Users/tutor/SPL Tests/tree.xml
Scope analysis OK
Type analysis OK - written to /Users/tutor/SPL Tests/tree.xml
Code generation OK - written to /Users/tutor/SPL Tests/target.cpp
```

The last line confirms that every stage passed and gives the full path of `target.cpp`, which can now be compiled as described in section 6.

Lexical error

```
Lexing error at line 3, column 7: token recognition error at: '@'
  print @oops ;
        ^
```

Syntax error

The `hint` line lists what the parser expected at that point:

```
Parsing error at line 4, column 6: missing '{' at 'print'
  hint: expected '{'
  then print "zero"
       ^
```

For lexical and syntax errors, line and column numbers start at 1, and the ^ marker points to where the error was found.

Semantic error

The program declares #x but prints #y. Parsing succeeds, so `tree.xml` is still written before the error is reported:

```
Parsed OK: undefined_var.txt
Parse tree: (spl_prog (p (v_decl #x v_decl) : f_decl : (algo (instr print (outp ( (term #y
termTail) ))) ; algo)) <EOF>)
Outputting the XML now
Check /Users/tutor/SPL Tests/tree.xml
Semantic error: variable #y is used but never declared (tree node 14)
```

The `tree` node number is the `id` of the offending node in `tree.xml`.

Type error

The program assigns `mod ( 5 2.5 )`. Scope analysis passes, then type checking rejects the decimal number:

```
Parsed OK: mod_with_decimal.txt
Parse tree: (spl_prog (p (v_decl #x v_decl) : f_decl : (algo (instr #x (instrTail = (term
mod ( (term 5) (term 2.5) )))) ; algo)) <EOF>)
Outputting the XML now
Check /Users/tutor/SPL Tests/tree.xml
Scope analysis OK
Semantic error: A float-integer-conflict might perhaps be possible
  the number 2.5 (line 1 (node 19)) has a decimal dot,
  but the program uses mod at: line 1 (node 14)
  With mod, every number in the program must be an integer.
```

Code generation error

Function `#f` takes two parameters but is called with one argument. The argument count is checked when the call is inlined, so scope analysis and type checking both pass first:

```
Parsed OK: arg_count.txt
Parse tree: (spl_prog (p (v_decl #x v_decl) : (f_decl (f_type num #f ( (v_decl #a (v_decl
#b v_decl)) ) { (p v_decl : f_decl : algo) return ( (term add ( (term #a termTail) (term #b
termTail) )) ) }) f_decl) : (algo (instr #x (instrTail = (term #f (termTail ( (input (term
1) input) ))))) ; algo)) <EOF>)
Outputting the XML now
Check /Users/tutor/SPL Tests/tree.xml
Scope analysis OK
Type analysis OK - written to /Users/tutor/SPL Tests/tree.xml
Code generation error: Function #f expects 2 argument(s) but was called with 1
```

Input file not found

```
Input file not found: /Users/tutor/SPL Tests/tset1.txt
Usage: java -jar group-21.jar <path-to-SPL-file>
```

The message shows the full path the program tried, which makes a misspelt name or wrong folder easy to spot.

9. Code generation notes (Phases 3a and 3b)

Function and procedure calls (inlining)

SPL functions and procedures are not translated into C++ functions. Instead, each call is replaced by a copy of the callee's body:

1. The arguments are evaluated from left to right, in the caller's scope.
2. The parameters are declared and then assigned the argument values inside the inlined block.

3. The callee's body is regenerated from the syntax tree with fresh temporary variables and labels, so that two calls to the same function never share names.
4. For a `num` function, the return value is copied into the caller's variable that receives the result.

Rejected call usage

- A `void` procedure used as a value inside a term (type checking)
- A `num` function used on its own as an instruction (type checking)
- A call with the wrong number of arguments (code generation, when the call is inlined)
- Recursive calls. Inlining copies a function's body into each call site, so a function that calls itself would never stop expanding. Recursion is therefore rejected during semantic analysis.
- Calls from inside a function to a function declared at an outer scope level. For example, a call to `#factorial` from inside `#factorial` is reported as: `Semantic error: function #factorial is not listed at scope level 1 (it is declared at level 0; recursion and calling outer functions are not allowed)`.

10. Design decisions

- **One numeric type per program.** If the program contains a decimal number or uses `div`, every numeric variable and temporary in `target.cpp` is a `float`, so `div` always divides exactly (`div ( 7 2 )` gives `3.5`). Otherwise all are `int`. Because `mod` only works on integers, a program cannot use both `mod` and `div`.
- **Two-label translation of conditions.** Every condition is translated with a true label and a false label, which gives correct short-circuit evaluation of `and`, `or` and `not`. In an `if … then … else`, the else part is placed directly after the condition, so for a simple comparison such as `eq` the false case is reached by falling through rather than by a jump, and the else label has no matching `goto`. This is why g++ reports unused labels when `-Wall` is used.
- **Loop comments follow the translated form.** A `while` loop is translated as `until not(condition)`, so its comments and label order use until-form markers; a `do … until` loop correspondingly uses while-form markers.
- **Code-mine sanitising.** The code-mine is documentation only, and inner comment markers are rewritten so that it remains one valid outer comment.

11. Troubleshooting

Message or symptom	Cause	Fix
`'java' is not recognized` (Windows) or `command not found: java` (macOS)	Java is not installed, or not on the PATH	Install Java 21 or newer, then open a new terminal
`UnsupportedClassVersionError`	The installed Java is older than 21	Install Java 21 or newer
`Unable to access jarfile group-21.jar`	The terminal is not in the folder containing the jar	Change to that folder with `cd`, or give the full path to the jar

Message or symptom	Cause	Fix
`Input file not found: …`	The test file path is wrong	Check the spelling and folder; quote paths that contain spaces
`g++: command not found` or `'g++' is not recognized`	g++ is not installed, or not on the PATH	Install g++ as described in section 3
`g++: error: target.cpp: No such file or directory`	The SPL program was rejected, or the terminal is in a different folder	Check the compiler's output for an error; run g++ in the folder where the compiler was run
g++ warnings about unused labels	`-Wall` was used	Harmless; compile without `-Wall` (see section 10)
Double-clicking the jar does nothing	It is a console program	Run it from a terminal as described in section 5

12. Declaration 1: Submission class

We, the undersigned members of Group 21, jointly declare that this submission is made as a:

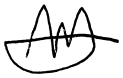
- ☐ 2nd Class submission (maximum 3/6: semantic analysis and type checking only)
- ☒ **1st Class submission (maximum 6/6: including executable target code generation)**

All members agree to this selected category.

Full name	Student number	Signature	Date
Antony Van Straten	24590739		25/10/2026
Keegan Lewis	24594522		25/10/2026
Aidan Dawson	24593542		25/10/2026
Sofia Finlayson	24593240		25/10/2026

13. Declaration 2: Awareness and consent

We, the undersigned members of Group 21, jointly declare that each member is fully aware of the contents of the submitted ZIP package and agrees with submission of this package in its submitted form.

Full name	Student number	Signature	Date
Antony Van Straten	24590739		25/10/2026
Keegan Lewis	24594522		25/10/2026
Aidan Dawson	24593542		25/10/2026
Sofia Finlayson	24593240		25/10/2026